



**AiCE Undergraduate Research Project
Final Report**

Spring 2024 Semester

AI-driven Traffic Management for a Better Bangkok

Team Members

Anapat Bunyapitaktamrong [Aut]

Chan Bormei Suy [Peach]

Inkaphol Siriwongwattana [Ink]

Nathaphob Aroonpairoj [Conc]

Paksaran Kongkaew [Minnie]

Sira Kongsiri [Mai]

Advisor

Dr. H. Miri

05/05/2024

Table of Contents

Chapter 1 Introduction.....	3
1.1 Problem Statement.....	3
1.2 Proposed Solution.....	3
1.3 Project Objectives.....	3
Chapter 2 Background.....	5
2.1 Fundamental Theory and Concepts.....	5
2.1.1 Digital Image Structures.....	5
2.1.2 Image Operations.....	6
Resizing.....	6
2.1.3 Artificial Neural Network or Forward Feed Neural Network.....	6
2.1.4 Backpropagation.....	7
2.1.5 Transfer Learning.....	7
2.1.6 Convolutional Neural Networks.....	7
Overview.....	7
Convolutional Layers.....	8
Pooling Layers.....	10
ReLU Layers.....	11
Fully connected Layers.....	11
2.1.7 Kalman Filter.....	11
2.2 Technologies.....	11
2.2.1 Pytorch.....	11
2.2.2 YOLO.....	12
2.2.3 Samyan Smart City.....	13
2.2.4 OpenCV.....	13
2.2.6 Apex.....	13
2.2.7 Roboflow.....	14
2.2.8 SAHI (Sliced Aided Hyper Inference).....	14
2.2.9 DeepSORT and Re-Identification.....	14
2.2.10 Apache Kafka.....	15
2.2.11 Log Stash.....	15
2.2.12 ElasticSearch.....	15
2.2.13 SUMO.....	15
2.2.14 React JS.....	16
Chapter 3 Methodology.....	17
3.1 Pipeline Construction.....	17
3.2 Model Connection.....	18
3.3 SUMO Simulation.....	18
3.4 Visualization Dashboard.....	19
Chapter 4 Results.....	20
Chapter 5 Conclusion.....	23

Chapter 1 Introduction

1.1 Problem Statement

A constant source of grief on city roads, traffic is a fact of life for city workers, with many forced to endure the same monotony for sometimes hours at a time. Bangkok, the capital city of Thailand, faces this issue constantly, being ranked as the 52nd most congested city in the world. While not as dire as cities such as London or New York, if filtered the rankings by how much the problem has worsened from 2021, Bangkok's rank rose significantly placing it in second after Jarkarta; this is illustrated when comparing the 2023 and 2021 indices, the time it takes to travel 10 kilometers in Bangkok has risen by around two and a half minutes, which is almost a full minute more than that of London's [1]. Although a small amount of congestion can signify a growing economy, too much restriction in the flow of traffic can lead to a decrease in employment growth rates, as longer travel times can act as a deterring factor when considering employment [2].

While we recognize that large-scale improvements to commuting time can likely only be achieved by changing the layout and routing of the roads themselves, this is much harder to implement in practice as houses, apartments, businesses, and communities already have long-established roots around the current road network. Therefore, if the previously mentioned method is not practically possible on a large scale, one of the means of improvement left is to optimize the existing network. AI-driven Traffic Management for a Better Bangkok (ATMBB) aims to act as a solution.

1.2 Proposed Solution

When finalized, our project aims to create a traffic signal calibration system which takes footage from the native ecosystem of surveillance cameras, map them, label them, and pass them through a Machine Learning model to glean information about the given footage; this will provide us with the number of vehicles queuing on each side of the intersection. Subsequently, this information will be used as data for another model which will calculate the optimal signal timings for the range of intersections. A visualization dashboard will be set up, so that the current timings and detections can be observed by the relevant controlling bodies; this will also include heatmaps which will give the viewer a better sense of the traffic density of a particular area.

1.3 Project Objectives

After the object detection model's development in the last semester, we planned on constructing a back end pipeline, along with expanding the already present visualization dashboard into a more substantial component of the project. Furthermore, a signal

optimization program will be implemented using data gathered from the object detection model; this will include factors such as velocity, object count within desired zones, etc. The three components stated above will be integrated into a single system, with the model sending road data to the optimization program by way of the backend pipeline; this data will also be sent to the front end to provide information about current traffic conditions to the overseers.

Chapter 2 Background

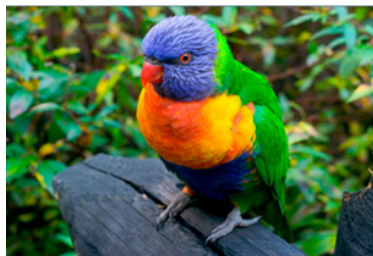
2.1 Fundamental Theory and Concepts

2.1.1 Digital Image Structures

A pixel serves as the tiniest unit in an image, containing data about its color intensity. Essentially, a digital image is a collective arrangement of pixels. The image quality is determined by the number of pixels it contains, a concept known as resolution. Higher pixel density results in better quality, offering more detailed visuals. However, this increased resolution also comes with drawbacks, such as longer processing times and greater disk or memory space requirements.

Digital images fall into two main categories: grayscale and color. The intensity of a color in any image is represented by a numerical value ranging from 0 to 255, typically stored as an 8-bit number. A channel is a grayscale image matching the size of the original image and linked to one of the primary colors. In grayscale images, pixel intensities range from 0 being black to 255 being white with the image having a single channel. In contrast, color images consist of three channels corresponding to red, green, and blue, superimposed to create the final image viewed by the user.

[3]



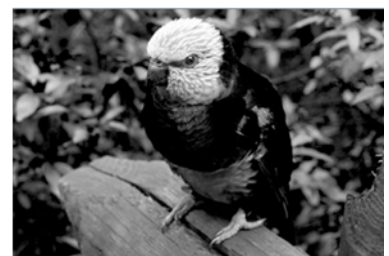
Original Image



Red



Green



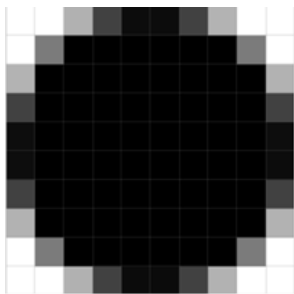
Blue

2.1.2 Image Operations

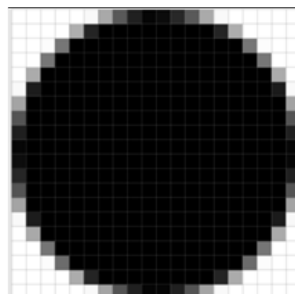
Resizing

The process of resizing involves adjusting the number of pixels in an image, either increasing or decreasing it. There are two approaches to this adjustment: upscaling, which increases the image's resolution, and downscaling, which decreases it. It's important to note that simply increasing the pixel count does not guarantee an improvement in quality, especially if the original image quality is low, as in such cases, a method called nearest neighbor resampling is applied during the upscaling or downscaling process. Nearest neighbor resampling involves incorporating the colors of neighboring pixels into a newly formed pixel. This occurs when new pixels are created during upscaling or when existing pixels are merged during downscaling.

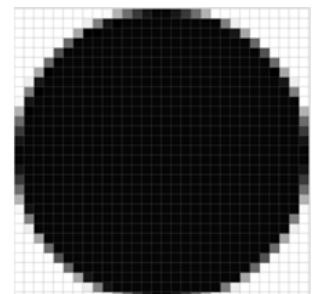
[4]



(10 x 10px)



(20x 20px)



(30 x 30px)

2.1.3 Artificial Neural Network or Forward Feed Neural Network

Artificial Neural Networks (ANNs) derive their name from their operational mechanism, which involves fine-tuning interconnected neurons within their system. The intricate network of neurons resembles one found in a brain, and ANNs are particularly effective in handling substantial datasets, excelling at pattern recognition and disregarding minor inconsistencies. The architecture of an ANN comprises an input layer, multiple hidden layers, and an output layer, through which input data traverses in sequential order. This unidirectional flow of data has led to ANNs being referred to as Forward Feed Neural Networks. While there is a single input and output layer, the researcher has the flexibility to determine the number of hidden layers, which is not predetermined. Owing to its structure, ANNs can process any data that can be converted into a numerical format. In applications involving image data, the hidden layers of an ANN detect specific features within the images. For instance, one layer may identify lines, while others focus on attributes such as shadows. Neurons are interconnected through a web of connections, each associated with a numeric weight. Since neurons in a layer typically connect only to immediate neighbors in adjacent layers, the input and output layers remain indirectly linked.

User input initiates the process, entering the network through the input layer. The input undergoes weighting before progressing to the first hidden layer, and this weighting process repeats through subsequent layers until the transformed input reaches the output layer, providing the model's prediction. Notably, the number of input and output neurons need not be identical. Input parameters represent different user-provided inputs, while output parameters signify the potential results anticipated by the user.

2.1.4 Backpropagation

Loss or error in ANNs is calculated through a process called backpropagation. This process involves computing the loss by starting from the output layer and iteratively moving backward through each layer to the input layer. The loss is determined by comparing the model's predicted value to the actual answer it was expected to generate. If these values differ, the loss is non-zero, and its magnitude increases proportionally based on the deviation from the target. Subsequently, the calculated loss is employed to adjust the weights and biases of each layer. This iterative correction aims to enhance the model's accuracy in the following cycle, minimizing the loss generated by the difference between the predicted and actual values.

2.1.5 Transfer Learning

Transfer learning operates on the idea of repurposing a pre-trained model designed for a specific task to address a similar task. As the model already possesses some level of proficiency in recognizing patterns necessary for task completion, it significantly reduces the time and resources required for re-learning concepts. Consequently, models employing transfer learning often achieve competitive solutions in a shorter time frame, utilizing fewer resources. This learning approach is particularly prevalent in pre-trained models employed for recognition or classification tasks, where leveraging a model already skilled in feature extraction substantially accelerates the emergence of satisfactory weight configurations.

2.1.6 Convolutional Neural Networks

Overview

Convolutional Neural Networks (CNNs) specialize in image recognition tasks and have a unique yet complex structure. They consist of a sequence of three primary layers: convolution, pooling, and ReLU layers, followed by one or multiple fully connected layers. The convolutional layers play a central role in the feature extraction process, capturing relevant features from the input images. Subsequently, these features undergo compression through max or min pooling, followed by processing through a ReLU layer, which eliminates non-positive numbers from the matrix. The resulting output is then fed

into fully connected layers that compile and map it to the corresponding output variable. Similar to traditional ANNs, CNNs leverage backpropagation, allowing the model to continuously refine itself based on previous predictions.

Convolutional Layers

The convolutional layer is named for its main process, which involves merging a filter with an input matrix to generate an output. A filter is a fixed-size matrix with individual cells each holding a value; these values' arrangement dictates the features the kernel will identify, such as corners, shadows, edges, or lines. In a standard Convolutional Neural Network (CNN), the initial kernel elements are typically set to random values. As the network learns, the kernel adjusts to recognize specific features. Kernel sizes are generally smaller than the images they process, handling the image one pixel group at a time. The process initiates from the top-left, with the filter moving right until it can no longer proceed, then shifting downwards. This repeats until the bottom-right of the image is reached. The speed of the filter's movement is determined by the stride, expressed as (x, y), where the first number denotes horizontal movement, and the second denotes vertical movement. Larger stride values lead to a smaller output shape and faster computation, however, this can possibly leave behind important identifying features.

In the context of a 7x7 image processed by a 3x3 kernel, the convolution considers the picture in 9-pixel groups, corresponding to the kernel size. Referring to Figure 2.1.6.1, and Figure 2.1.6.3 a-c, the kernel overlays each group of pixels, generating an output; this is calculated by summing the product of the filter's value at that pixel and the underlying pixel value for each pixel in the batch, repeating until the convolution covers the entire image at a stride length of 1. The resulting 5x5 matrix is then passed to the next layer for further processing or to a fully-connected layer for a prediction.

0.0	0.0	0.35	0.35	0.35	0.0	0.0
0.0	0.35	0.65	0.65	0.65	0.35	0.0
0.35	0.65	1.0	1.0	1.0	0.65	0.35
0.35	0.65	1.0	1.0	1.0	0.65	0.35
0.35	0.65	1.0	1.0	1.0	0.65	0.35
0.0	0.35	0.65	0.65	0.65	0.35	0.0
0.0	0.0	0.35	0.35	0.35	0.0	0.0

Original Image
[Figure 2.1.6.1]

1.0	-1.0	0.0
1.0	-1.0	0.0
1.0	-1.0	0.0

Kernel
[Figure 2.1.6.2]

AA	AB	AC				
BA	BB	BC				
CA	CB	CC				

[Figure 2.1.6.3a]

	AA	AB	AC			
	BA	BB	BC			
	CA	CB	CC			

[Figure 2.1.6.3b]

		AA	AB	AC		
		BA	BB	BC		
		CA	CB	CC		

[Figure 2.1.6.3c]

Kernel's Progression Through the Original Picture

0.0	0.0	0.35	0.35	0.35	0.0	0.0
0.0	0.35	0.65	0.65	0.65	0.35	0.0
0.35	0.65	1.0	1.0	1.0	0.65	0.35
0.35	0.65	1.0	1.0	1.0	0.65	0.35
0.35	0.65	1.0	1.0	1.0	0.65	0.35
0.0	0.35	0.65	0.65	0.65	0.35	0.0
0.0	0.0	0.35	0.35	0.35	0.0	0.0

[Figure 2.1.6.4]

1.0	-1.0	0.0
1.0	-1.0	0.0
1.0	-1.0	0.0

[Figure 2.1.6.5]

Kernel Overlaid onto the Original Image

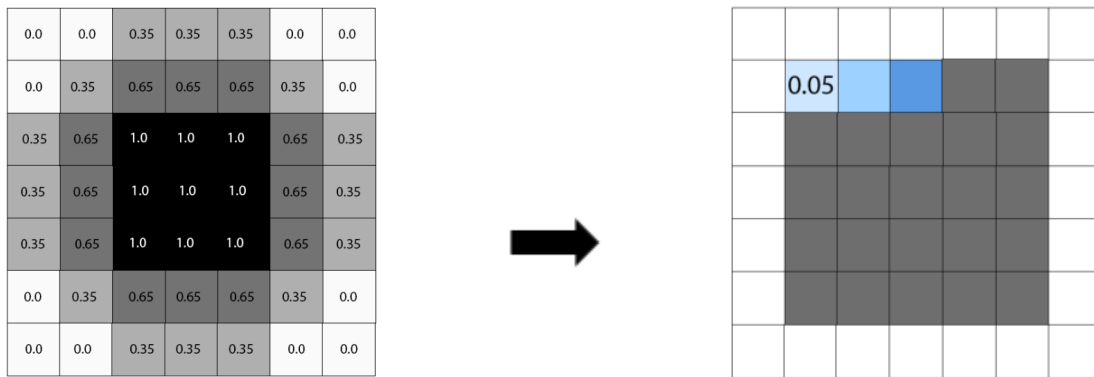
$$\text{Output Pixel} = 1.0[\text{AA}] - 1.0[\text{AB}] + 0.0[\text{AC}] + 1.0[\text{BA}] - 1.0[\text{BB}] + 0.0[\text{BC}] + 1.0[\text{CA}] - 1.0[\text{CB}] + 0.0[\text{CC}]$$

$$\text{Output Pixel} = 1.0[0.0] - 1.0[0.0] + 0.0[0.35] + 1.0[0.0] - 1.0[0.35] + 0.0[0.65] + 1.0[0.35] - 1.0[0.65] + 0.0[1.0]$$

$$\text{Output Pixel} = 0.05$$

Convolution Equation for the First Pixel

[Figure 2.1.6.5]



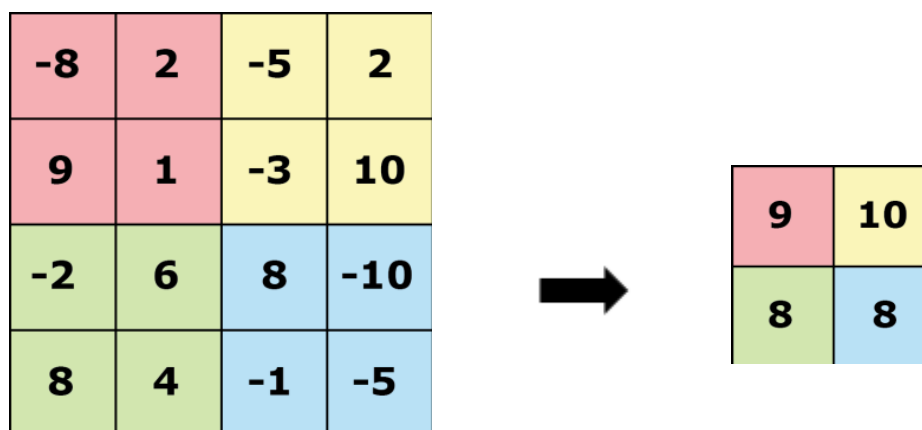
Convolution Overview

[Figure 2.1.6.6]

Pooling Layers

The concept of pooling revolves around compressing an image into a specific size while retaining important information. Out of the three main types of pooling, which include max-pooling, min-pooling, and avg-pooling, the one that is the most commonly used is max-pooling. Max-pooling gets its name from the fact that when the group of pixels are compressed into one, the resulting pixel will have the value of the highest value pixel within the previous group. Min-pooling operates on the same concept, but instead of being the maximum value, it is the minimum, and the average pool averages out each pixel's value. The process is used to stabilize the position of the feature or object inside the image, as without the pooling layer, slight alterations including shifting may produce a different feature map.

To illustrate, figure 2.1.6.7 shows an input picture of size 4x4 being processed through a max pooling layer of size 2x2. Each 2x2 section of pixels will be grouped together without any overlap, and the pixel with the high value within each group of 4 pixels will become the output pixel.



Max Pooling Process

[Figure 2.1.6.7]

ReLU Layers

The ReLU layer works to clean up the current image; it does this by selecting every negative number and setting its value to zero, while positive numbers stay as they were. The process is done to refine the output of the previous layer, usually a convolutional layer, as a negative number means that it has low relevance towards the identification of a targeted feature. Because of this ReLU layers often follow immediately after convolutional layers, as the ReLU layer will be able to catch any negative values before they enter into the pooling layers.

Fully connected Layers

The output from the pooling layer ultimately reaches the fully connected layer in the form of a 1-D vector. Given the fully connected nature of this layer, wherein neurons establish connections with every weight from the preceding layer, the resulting output reflects the collective output of every element from the entire input picture, rather than focusing on specific portions.

2.1.7 Kalman Filter

The Kalman Filter is an algorithm that infers the state of a noisy and dynamic system. The algorithm achieves this process by taking an input, in this case an video, as a set of matrices and observing the changes incurred by the object; these changes are then compared to the predicted changes calculated by the algorithm, the algorithm then adjusts itself based on the difference in the actual and predicted value. Being able to infer the position of objects inside a frame is indispensable when implementing functions such as re-identification, as there is always a possibility of the model losing track of a sighted object. Hence, the Kalman Filter's role is making sure that the object can be easily re-acquired as well as tracked throughout the frame.

2.2 Technologies

2.2.1 Pytorch

PyTorch is an open-source machine learning framework developed initially as a part of MetaAI by Adam Paszke, Sam Gross, Soumith Chintala, and Gregory Chanan [10]. The framework was initially released to the public in September 2016, with the models and their functionalities, such as the training and validation steps, being implemented as a set of Python classes. Its ease of use and readability compared to TensorFlow made it a solid choice for this project. Furthermore, this ability to easily import and use partially trained models in a process called transfer learning as shown in figure 2.2.1.1, which is highly useful for projects with small datasets such as ours. Additional capabilities include its ability to pre-process the images without the use of a third-party library; this being functions that resize, rotate, greyscale and add random noise to the input image.

However, in our current project, the main use of the Pytorch library revolves around its cuda functionalities which allows us to tap into the device's gpu cores for training; this allows us to shorten the training time of our constructed model. As YOLO is used as the base for our current project and since it's based on the ResNet model, the pytorch library will have to be installed for the underlying dependencies. [5]

2.2.2 YOLO

The YOLO model or the "You Only Look Once" model is a model created by the Ultralytics group in June of 2020 with the popular ResNet as a base with the specific goal of object classification and object localization in mind; this pairing combined enables the model to perform a process called object detection, which is the process of localizing then classifying multiple target objects inside a given image. [6] A brief breakdown of the YOLO process revolves around the shape of the output; this usually contains the detected object's bounding box, presence of an object, percentage of confidence. The bounding box in this case pertains to the location and dimension of the box the object is currently residing in, its location and size. In regular CNNs, the output's shape consists of a range of integers, but the output for a YOLO model is that of a 1D array due to the range of data it produces. Although this structure does come with its own set of limitations, since its output is just one array, it is not possible to detect more than one object. This is unless the image is broken down into a grid of cells with each of the cells acting almost as a subset of the larger image, each of these being able to detect another object. However, increasing the grid cells' dimensions will increase the number of processes the model will have to perform which will increase the time needed to perform the detection. Due to that reasoning, YOLO still has difficulty identifying large clusters of items closely packed together with the current maximum number of instances detectable inside a single frame, while still being viable time wise, is around 20.

To date, the 8th version is the newest model released by the Ultralytics team, but our team has decided to continue our development in the 5th version, since the 8th version is still quite unstable in its detection and also lacks troubleshooting documentation.

Furthermore, other versions that seemed to be a possible fit for our project at first, such as YOLONas which was supposed to be geared towards live video feed, failed to produce a result that was satisfactory in its evaluation. The reasons above culminated in the decision to continue our work on the 5th version of the model.

2.2.3 Samyan Smart City

Created as a project by Chulalongkorn University and overseen by the Property Management of Chulalongkorn University (PMCU), the Samyan Smart City project aims to create and improve the ecosystem around the Samyan area of Bangkok; this involves projects centered around energy infrastructure, transportation, and the digitalization of the surrounding area. The dataset used in this project contains data from two intersections and was kindly provided to us by the PMCU. [7]

2.2.4 OpenCV

A popular computer vision library first developed in 1999, OpenCV was first developed by Intel, specifically Gray Bradsky who was a computer vision engineer employed by Intel at the time, and was subsequently released in 2000. The library consisted of a wide range of functionalities including image processing, feature detection, object detection, and displaying data. As for our project, this library will be used solely for their output display and framing functionalities, as it is employed to showcase our visualizations such as the heatmap and crop the mapping so that it only displays objects on the roads.[8]

2.2.6 Apex

Apex is a supercomputer and storage platform designed explicitly for processing intensive activities like training a complex machine learning model. It is currently hosted at CMKL University itself and contains Nvidia DGX A100 48x GPUs, which have a high amount of processing power, and a Tensor core GPU which is more optimized for training when compared to a Cuda core GPU which is standard on most computers. Suppose multiple models are assigned to run at once. Instead of splitting the processing power of one GPU to run several models, which would happen if a laptop is used, each of the models will receive its own GPU core, drastically decreasing the time needed to train multiple models. Apex's processing power was used to consecutively train variations of our ResNet model, which made it possible for us to experiment and make minute adjustments to our training variables, as it shortened the training time needed to complete all the training cycles significantly.

2.2.7 Roboflow

Roboflow is a computer vision labeling website that is geared towards creating a streamlined way to produce datasets for computer vision models. Venture funded and released in 2020, Roboflow has supported a wide range of frameworks including ResNet, TensorFlow, and RoboFlow. The last one stated is of importance as that is the current basis of our project. Using Roboflow, we have been able to separate, annotate, and export datasets to our models through one platform, which greatly decreased the complexity of the preparation process. [9]

2.2.8 SAHI (Sliced Aided Hyper Inference)

Developed and packaged as a part of the Ultralytics library, SAHI was made to process images or videos with higher resolutions by dividing, identifying, and stitching back together the resulting output. The benefit this system has over the normal identification process pertains to the overall memory utilization. While using the SAHI system, the process works with small sub-sections of the given image which is much less memory intensive than processing the entire image all at once, especially if the image is of high resolution. Furthermore, as all of the sub-images are treated as individual frames, the quality of detection is also often higher using the SAHI system as the model is no longer under a high amount of computational strain; this feature is extremely advantageous to our project since it makes it possible for model to accurately count the number of road bound vehicles even while clustered together. [10]

2.2.9 DeepSORT and Re-Identification

An extension of the SORT, which is an abbreviation of the phrase Simple Online Realtime Tracking, DeepSORT further improves on the re-identification abilities of its predecessors. However, before the functionalities of DeepSORT can be fully explained, importance must first be given to how the normal SORT algorithm works. The SORT algorithm works by way of the Kalman Filter which has been explained in the previous section. In short, the location of the object in the current and subsequent frames are documented and compared against an estimation made by the algorithm; this comparison is made until the estimation model starts to mimic the movement patterns of the actual subject. Through the use of this process along with logging the number of objects currently on screen, we can achieve a rudimentary form of what can be described as object tracking and re-identification. The way that DeepSORT builds upon this functionality is by improving the re-identification aspect of the algorithm; this is done by factoring both the object's physical attributes along with its velocity and bearing it into consideration when performing identification; this significantly increases the tracking accuracy along with the retention of each object's identification number. [11]

2.2.10 Apache Kafka

Created by the same developers of the Apache Server Platform, Apache Kafka serves as a data distribution framework: meaning that it is a counterpart to the REST framework. Its distribution method is what separates the two as the REST API operates based on a request and response model, while Kafka works of a producer and consumer model; this main difference is why Kafka was used in this project instead of REST, as the model it uses is more suitable for large streams of data, as it doesn't need to send any type of request message which in turn lowers the system's latency. Although, the balancing factor to this increase in speed is the additional set-up the Kafka system needs to send and receive information, as the broadcast and listening nodes need to be manually installed at each of the component's locations before data transfer can begin.

2.2.11 Log Stash

Log Stash works as a data filtration component packaged with ElasticSearch, which in this case will act as our server side database. The reason this is needed even though data can be directly piped into ElasticSearch using Kafka is due to Log Stash's range of provided services: fault tolerance, input multiplexing, and data processing. Fault tolerance comes in the form of "Persistent Queues", which is a functionality that stores yet to be processed data; this means that if the Log Stash instances are taken offline, the queuing data will be preserved within the system. These instances also provide a way for the database to receive information from multiple sources at the same time, which will be needed as the number of camera instances grows in scale.

2.2.12 ElasticSearch

Acting as our database, ElasticSearch acts in tandem with Log Stash to create our data storage and management system; these systems were chosen because our project would have to process a large amount of data, and would need to constantly query data for various uses. The reasoning above was why a simpler system like SQL was not considered. Elasticsearch's main draw is mainly its ability of inverted indexing, meaning the framework catalogs and assigns unique keywords to each of the objects in its database. The keywords are now identifiers which can be used during searching, giving Elasticsearch a distinct advantage as it does not need to go through all the content within each of the objects but rather just filters out the possible indexes using these keywords. As this project grows, the program's sharding abilities will also be an easy way to scale up the database's size since instances are completely independent of each other and therefore more instances can later be grafted onto the system without the need for data porting.

2.2.13 SUMO

SUMO is a traffic simulation program created by the German Aerospace Center, made to simulate traffic movements on a large scale; this program proved to be directly applicable to our project, as we could choose from a selection of road formats and rules of law without the need of developing them from scratch. The program also provides a way

for us to insert our data into the program's decision making progress, and also to calibrate some of the on-road variables: vehicle ramp up speed, maximum across intersection speed, etc. The program also provides a simple yet effective simulation of the intersection as the program itself runs, providing developers with the chance to spot any possible edge cases which may not present itself in purely statistical data outputs.

2.2.14 React JS

React is a Javascript framework usually used when developing user interfaces, placing importance on the reusability of each of the code components and user experience. Code reusability is achieved in the form of components, as each object within the interface can be packaged up into easily used bundles called components; these can be used with a functionality called properties to further standardize the development process as everything inside these components can be substituted with variables which in turn can be linked to the property names given to the components. As React is a Javascript library, it can also perform selective rendering, meaning that the website can choose to only re-render objects that experienced any changes; this increases usability significantly since it lessens the amount of data request calls, which progresses to a decrease in the amount of objects the client needs to retrieve on average. Our dashboard had a number of overlapping objects, especially the header and footer regions of the page. Furthermore, due to the complexity of some of the pages the selective rendering assists greatly in keeping loading and latency to an acceptable level.

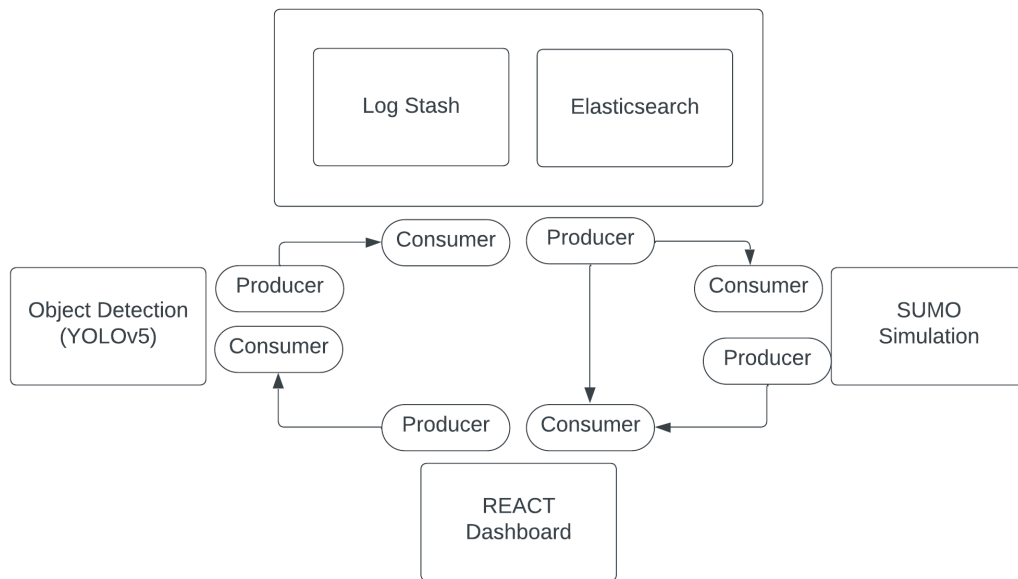
Chapter 3 Methodology

The methodology section will be a continuation of the work done within the last semester, meaning that the architecture and development of the YOLO based object detection model won't be discussed here as it has already been explored in detail within last semester's project report; this section therefore will only contain information on the process completed within the second semester: pipeline construction, model connection, and SUMO application.

3.1 Pipeline Construction

To preface this section, at the beginning of the semester, our group was looking towards using NVIDIA's premade services instead of developing our own backend pipeline, their Metropolis Microservices to be exact, however this was not possible; the reason for this shift in direction will be explained in the "Issues and Obstacles" section of the report, as its explanation will subtract of the chapter's intentions. Our first action when it came to the construction of this pipeline was the research into a whole range of different tech stacks and ecosystems and weighing out the advantages and disadvantages of each of the encountered systems. Our system would be handling large amounts of data which meant that a REST based system would not be possible as it would introduce latency into our system. Kafka was instead chosen for its producer and consumer model; this premise revolved around the use of producer and consumer nodes. The first being island nodes which would constantly be sent data, regardless of the amount of listener nodes, and the latter would tap into these nodes and receive the broadcasted information. Node based models are much faster because they aren't bound by the request and response model of REST. Given that the transfer speed between the two systems are the same, the node based format would be twice as fast as the request based model due to this fact. Other technologies used in this system were chosen around the fact that the system itself was expected to handle large quantities of data from multiple sources concurrently.

Our pipeline would have four distinct sections: database, object detection, front end, and SUMO; this means that producer and consumer nodes were installed in each of these components, which served as a connection point between the four modules; this would be done through Kafka's Zookeeper functionality which provides a centralized way to keep track of the node's status, construction, and termination. Lastly, the consumer nodes will also have to be connected to the appropriate producer's broadcasting port. For example, the database component's consumer node will have to be connected to the object detection's producer node so it can receive and later process the transferred data.



Pipeline Overview

[Figure 3.1.1]

3.2 Model Connection

After the nodes' installation, the first functionality that needed to be completed was the object detection model's output repackaging; this process would see the output data re-packaged into a form that could be taken in for storage by Elasticsearch. Output data is first initialized in the form of variables, which are then passed to a Kafka producer script; this list of information will then be reformatted into JSON files for transportation. After the conversion, the package is broadcasted by the producer node to any consumer nodes currently listening to it, which in this case would be the one associated with the database. Within the database component, the message would then be transported to Log Stash through a Kafka plug-in, which will then finally be transported to Elastic search for storage.

3.3 SUMO Simulation

In our current state, the SUMO section of our system has yet to be fully integrated within the Kafka pipeline at large, however, we were able to test out our signal optimization on mock data provided by the SUMO program itself; this data can and will be switched with the road data gleaned from the object detection model in the future. The default algorithm for signal follows a set repeating pattern with no concern given to any other variable. So, instead of the given system, the first iteration of our signal optimization is based strictly on the car's time in zone and the number of cars there are in the region of interest. We also had to change a number of variables within the SUMO application itself like which we found skewed the resultant time. The two most notable ones in this case were the maximum speed when crossing the intersection and the time needed for a vehicle to reach said speed. After our first round of testing, we discovered that the vehicles themselves were moving uncharacteristically faster than normal and their ramp up time was incredibly short; this over-inflated the time decrease between our's and the base

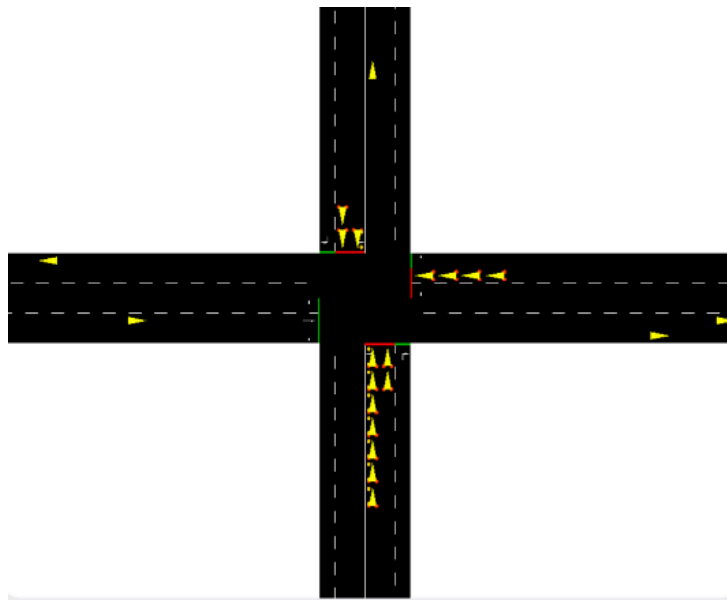
model, which at the start was an unrealistic 52%. However, this figure has dropped down to a more realistic level after the change.

3.4 Visualization Dashboard

Our work on the visualization dashboard this semester has been to transform our previous dashboard into a more standardized and user ready version, as our first implementation was mainly used as a platform to display our progress and was not user friendly. The main segment of development was completed using React JS with the repository itself being hosted on Github for easy distribution to the other members. The pages themselves mostly followed a strict layout with the same header and footer, so the layout itself was the first to be drafted and designed. In the current version, most of the visualizations, such as the average velocity and car count, were moved to a page which displayed the camera's current details. However, the heatmap was unable to be retained as we found that the visualization had some significant issues. For example, in some instances, hot spots would appear and refuse to dissipate, causing a perpetual hot spot within the map. Due to that, the heat map will have to be removed from the dashboard until a fix to the issue can be found.

Chapter 4 Results

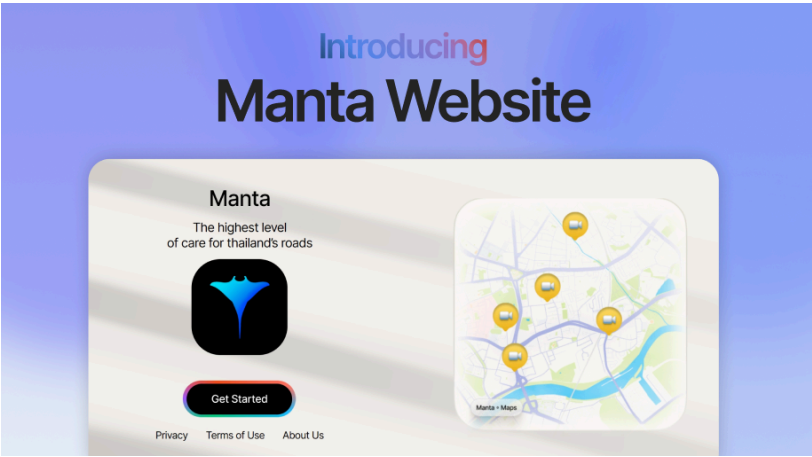
The pipeline between the database and object detection components were successfully implemented, with the database itself able to support the stream of data derived from the object detection model in near real time. As for the SUMO simulation, as stated before the first iteration of the custom algorithm was measured to be 52% faster than the base version. However, this figure was extremely misleading as some of the default variable values were unrealistically set. After the alteration, the final value measured was still a respectable 24% decrease in wait time; this was done using the same test case on both the base and our algorithm, yielding an average wait time of 19.2 and 14.6 seconds respectively.



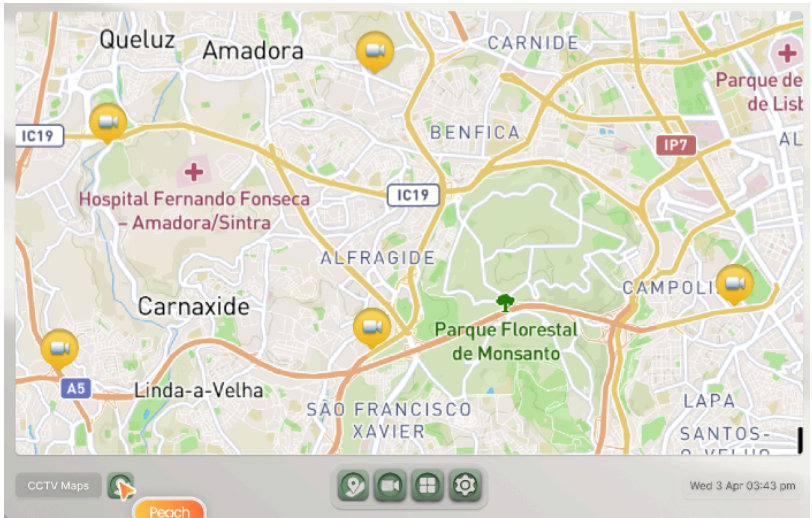
SUMO simulation while testing

[Figure 4.1]

The current iteration of our visualization dashboard consists of four main pages: landing, map overview, camera configuration, and camera statistics; this will be shown visually through Figures 4.2-5. The maps overview page, shows the location of each of the cameras that are currently in service, and is toggleable between three main view modes: satellite, default, and terrain. The CCTV configuration page is where the user can change various settings related to the target camera; this could be its region of interest or the location of the camera itself. Groupings can also be made for clusters of cameras that are near each other for easy access. Lastly, the camera statistics page shows the road data for that specific camera, such as the car count, velocity, and a live feed; this also includes a comparison between current data and the monthly or yearly average to provide the viewer with contextual information about the current flow of traffic.



Landing page
[Figure 4.2]

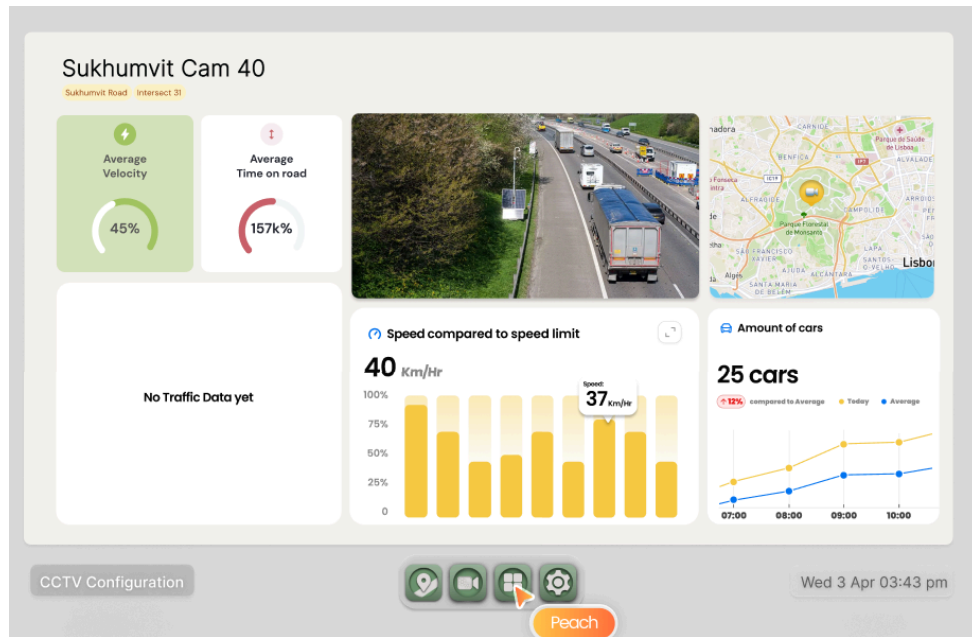


Map Overview Page
[Figure 4.3]



Camera Configurations Page

[Figure 4.4]



Camera Statistics Page
[Figure 4.5]

Chapter 5 Conclusion

5.1 Summary of Accomplishments

We were able to construct a data pipeline within our service's backend, and are currently using it to send data from our object detection module to our database. A traffic simulation program has also been set up, providing us a way to simulate and visualize our algorithm and any changes to it. Lastly, the visualization dashboard has been re-developed into a more user friendly and fleshed out component of our system.

5.2 Issues and Obstacles

At the beginning of the semester, we were planning on adopting NVIDIA's Microservices, as it was the only solution to our team's knowledge of the issue that had been persistent throughout the first semester; that being stable re-identification. Communications had been sent to NVIDIA requesting entry to the early access program, which took roughly 2-3 months. However, the obstacle in this case was the fact that to run the services, the group would need to use an extremely powerful computer, which was where APEX was supposed to fit it architecture wise; this however would not be realized as we would need root access to an entire node within APEX itself. Upper management was not willing to offer this up free of cost and responded back with a six-figure quote for basically renting out the node. There was an incident over this and eventually root access was given to the group, although, the group was no longer confident in the stability of the APEX platform; this lead it to becoming a test bed for some of the system's functionalities, however, the bulk of the remaining systems would have to be hosted on another system. The migration then presented another problem, as the microservices still required a number of GPUs to function which would not be possible in any normal computing system. GPU compartmentalization was the closest we ever got to a solution but the services would run so slowly and with so many lag spikes, the services themselves were unusable; this lead to the decision of just constructing our own services, as we had only one month left at that point and every thing we tried were eventually turning up as unsuccessful attempts at a problem that we had too little time to solve and too few resources to directly tackle.

5.3 Future Directions

As stated within the presentation, we're currently planning on continuing to work on three main aspects of the project: model fine-tuning, peer comparison, and camera positioning. We of course believe that further training would be beneficial to our YOLOv5 model, especially during edge cases. In the case of the SUMO simulation, the algorithm currently is quite basic and could use a significant amount of polish. If we can get some

output data from some of the other models currently on the market, we'd like to do some peer comparisons between our model and theirs'. Finally, giving more time and funding, we'd like to either reposition already existing cameras or install new ones entirely to provide the model with better viewing angles. We had been experiencing a few issues with deriving a stable vehicle velocity from cameras with sub-optimal road viewing angles; we think that this will be the key to solving that particular obstacle.

5.4 Lessons Learned

The main takeaway from this semester was to always have a second plan to anything you're planning to do. We were overly fixated on boarding NVIDIA's services, even though there were signs early on that there's a high likelihood that the services won't be usable. We bullishly tried every possible way to make it work, and tried something else when it didn't; this repeated until we were forced to abandon the concept with a month left on the schedule. While we were able to produce a number of our originally intended goals, the workload and stress negatively affected a large majority of our members. The other one would be that direct on-site communication would always beat online based forms. Our posters and slides were made possible because we pulled most of our members on-site for the last week of preparation; this made coordination and work partitioning much more straightforward as direct requests could be made for illustrations, data points, and percentages.

References

1. TOMTOM TRAFFIC INDEX Ranking 2022 (2023) TomTom traffic index – live traffic statistics and historical data. Available at: <https://www.tomtom.com/traffic-index/> (Accessed: 06 September 2023).
2. Sweet, M. (2013) 'Traffic congestion's economic impacts: Evidence from US metropolitan regions', *Urban Studies*, 51(10), pp. 2088–2110. doi:10.1177/0042098013505883.
3. Saponara, S. and Elhanashi, A. (2022) "Impact of image resizing on deep learning detectors for training time and model performance," *Lecture Notes in Electrical Engineering*, pp. 10–17. Available at: .
4. Inc., A. Images, Images - Foundations - Human Interface Guidelines - Design - Apple Developer. Available at: <https://developer.apple.com/design/human-interface-guidelines/foundations/images/> (Accessed: November 22, 2022).
5. Warsaw, A.P.U.of et al. (2019) PyTorch: Proceedings of the 33rd International Conference on Neural Information Processing Systems, Guide Proceedings. Available at: <https://dl.acm.org/doi/10.5555/3454287.3455008> (Accessed: December 8, 2022).
6. Jiang, P. et al. (2022) 'A review of Yolo algorithm developments', *Procedia Computer Science*, 199, pp. 1066–1073. doi:10.1016/j.procs.2022.01.135.
7. CU, P. (2023) Samyan Smart City – livable city for people, SAMYAN SMART CITY. Available at: <https://www.samyansmartcity.com/th/> (Accessed: 12 December 2023).
8. Brahmabhatt, S. (2013) 'Introduction to computer vision and opencv', *Practical OpenCV*, pp. 3–5. doi:10.1007/978-1-4302-6080-6_1.
9. Give your software the power to see objects in images and video (no date) Roboflow. Available at: <https://roboflow.com/> (Accessed: 12 December 2023).
10. Haider, N. and Mehta, A.K. (2022) 'A novel approach for small-object detection based on fine-tuning and sliced inference', 2022 2nd Asian Conference on Innovation in Technology (ASIANCON) [Preprint]. doi:10.1109/asiancon55314.2022.9909175.
11. Hou, X., Wang, Y. and Chau, L.-P. (2019) 'Vehicle tracking using deep sort with low confidence track filtering', 2019 16th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS) [Preprint]. doi:10.1109/avss.2019.8909903.